

归档聊天功能解释

会话 ID: 6a797ff5-dc64-83ed-a178-31d70b82a312 开始时间: 消息数: 10

用户 2026-08-10 09:38:47

archive chats 是干嘛的?

助手 2026-08-10 09:38:54

在 ChatGPT 里, **Archive chats (归档聊天)** 主要就是把旧对话从左侧聊天列表里收起来, 让界面更整洁, **但不会删除聊天内容**。

归档之后, 这些聊天:

- 不再显示在普通侧边栏聊天列表中;
- **仍然保存在你的账号里**;
- 仍然可以通过 ChatGPT 的搜索功能搜到;
- 可以在 **Settings → Data Controls → Archived Chats → Manage** 中查看, 并随时 **Unarchive (取消归档)** 恢复到正常聊天列表。

它和 **Delete (删除)** 差别很大: Archive 类似 “收进文件柜”, Delete 则是删除聊天, 而且已删除的聊天不能通过界面恢复。

比如你现在有很多论文、Stage 1 / Stage 2、毕业流程、邮件修改等聊天, 已经结束或者暂时不用的, 可以 Archive; 这样侧边栏会清爽很多, 但之后需要时还能搜索回来。

另外有一点容易混淆: **Archive 不等于让 ChatGPT 忘记这段聊天**。

如果你愿意, 我也可以继续给你解释 **Archive、Delete、Temporary Chat、Memory 这四个功能到底有什么区别**。

用户 2026-08-10 09:44:08

不同电脑的同一个账号的 codex, 可以互相查看聊天记录吗?

助手 2026-08-10 09:44:18

如果你说的是 **Codex 桌面端 / ChatGPT 里的 Codex**, 那么答案是:

同一个 OpenAI/ChatGPT 账号登录在两台电脑上, 并不意味着所有 Codex 本地聊天都会自动在两台电脑之间完整同步。

可以简单理解为:

- | 情况 | 另一台电脑能否看到 |
- | ---|---|
- | 普通 ChatGPT 聊天 | □ 通常同步 |
- | 云端保存的 Codex 会话/任务 | □ 有些可以 |
- | Codex Desktop 与本机 repository 绑定的本地会话 | □□ 不保证同步 |
- | Codex CLI 本地 session | □ 通常属于本机 |

| 两台电脑登录不同 OpenAI 账号 | □ |
| 同一个账号，但不同 workspace | □□ 可能看不到 |

尤其是 **Codex CLI**，OpenAI 将它描述为运行在你电脑上的 coding agent，因此它和普通 ChatGPT 的云端聊天历史不是完全一样的概念。□cite□turn246434search3□

不过现在有一个很有用的新功能

OpenAI 已经支持 **Codex Remote Connections**。例如：

电脑 A

```
Codex App
↓
thesis / Python
↓
Allow other devices to connect
```

电脑 B / 其他设备

可以通过同一个 **ChatGPT account + workspace** 连接到运行 Codex 的电脑 A。官方明确要求两端使用相同的 ChatGPT 账号和 workspace。□cite□turn246434search14□

所以这和“把聊天记录复制到电脑 B”稍微不同，更像：

> **电脑 B 远程继续控制电脑 A 上的 Codex 工作。**

这对于代码项目其实更加实用，因为 Codex 需要的不只是聊天记录，还包括你电脑上的：

```
project/
├── stage1/
├── stage2/
├── trainer.py
├── model.py
├── dataset.py
├── results/
└── .git/
```

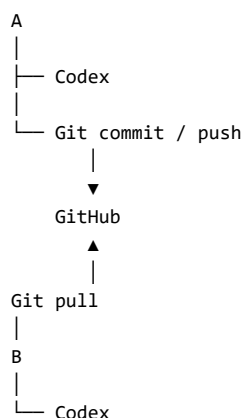
即使聊天同步到了电脑 B，如果这些文件不存在，Codex 也没办法原样继续分析这个项目。

如果你的目标是“两台电脑无缝继续同一个代码项目”

我更推荐：

GitHub + Codex + 同一个 ChatGPT 账号

例如：



这样 **代码状态由 GitHub 同步**，Codex 聊天负责解释和操作代码，可靠性会比单纯依赖聊天历史同步高很多。

而且 OpenAI 现在的桌面 App 正在把 **Chat、Work 和 Codex** 整合到一起；官方说明更新后的 App 会保留已有 Codex chats 和 projects，但这不等于所有“本机 Codex session”已经变成完整的跨电脑云同步。

如果你现在的实际情况是 **Windows 电脑 A 已经有很多 Codex 聊天，你想在 Windows/Mac 电脑 B 上继续这些聊天和同一个 GitHub 项目**，我可以直接告诉你**最稳妥的迁移/同步方法**，包括 `.codex`、GitHub 和 Codex App 分别该怎么处理。

用户 2026-08-10 09:49:39

我自己的电脑现在不在身边，Codex Desktop 与本机 repository 绑定的本地会话如何能让两个电脑保持记录同步？

助手 2026-08-10 09:49:56

可以，但需要区分 **“同步代码”** 和 **“同步 Codex 本地聊天线程”**。就你说的 **Codex Desktop + 本机 repository 的本地会话**，目前最关键的一点是：

“同一个 ChatGPT 账号登录两台电脑，并不会自动把这类本地 Codex 会话同步成两份。” Codex 的本地状态默认保存在本机 `CODEX_HOME`（通常是 `~/codex`），其中包括本地历史 `history.jsonl` 等；而 Codex 的历史也仍然与普通 ChatGPT 的云端聊天历史分开。

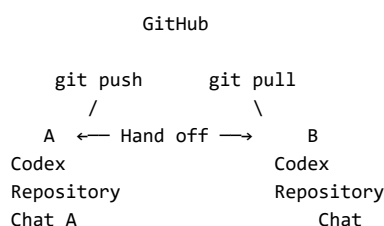
不过，**截至 2026 年 8 月，现在已经有一个比“复制聊天记录”更适合你的官方方案：Remote + Hand off。** Codex 可以在两台电脑之间把一个现有 chat 连同 Git 状态一起交接过去。

我最推荐你以后这样配置

假设：

A =
B =
GitHub =

整体结构：



具体做法是：

1. **两台电脑都 clone 同一个 Git repository。**

例如：

```
git clone <your-repository>
```

最好保持项目结构一致，例如：

```
thesis-code/
├─ stage1/
├─ stage2/
├─ model.py
├─ trainer.py
└─ .git/
```

2. **两台电脑都登录同一个 ChatGPT account + workspace。**

3. 在电脑 A 和电脑 B 的 ChatGPT/Codex Desktop 中配置：

```
Settings
↓
Connections
↓
Control this Mac/PC
Control other devices
```

Remote 官方支持访问另一台机器上的项目和现有聊天；宿主主机需要保持开机、联网并运行桌面 App。 [□cite□turn887245view1□](#)

4. 两台电脑上的 Codex 都保存**同一个 Git repository 对应的 project**。这是 Hand off 的必要条件；如果项目是 repo 的子目录，两边还需要保存同样的子目录。 [□cite□turn887245view1□](#)

5. 当你准备从 A 换到 B 时，在当前 Codex chat 底部选择：

```
Current run location
↓
Hand off
↓
Computer B
```

Codex 会在目标机器上创建或复用 worktree，并**传输 chat + Git state**，然后让这个聊天切换到目标主机继续运行。 [□cite□turn887245view1□](#)

这就非常接近你想要的：

> A 上聊到一半 → 去 B → 原来的 Codex conversation 接着聊。

而不是：

> A 有 conversation A
> B 又重新创建 conversation B
> 然后自己重新解释整个项目。

但是你现在有一个特殊问题：自己的电脑不在身边

这里分两种情况。

****情况 A：你之前已经给自己的电脑开启 Remote，而且它目前开机、联网、Codex/ChatGPT Desktop 在运行。****

那么有机会直接从你在这台电脑或手机 Remote 进去，继续原电脑上的 Codex chat。官方现在支持从另一台受支持的桌面设备继续工作；手机也可以通过 Remote 访问支持的 Codex chats。 [□cite□turn887245view1□turn887245view0□](#)

甚至可以进一步：

```
↓
Codex Chat
↓
Hand off
↓
```

前提是两边已经满足 Remote/项目匹配条件。

****情况 B：你以前没有开启 Remote，而且自己的电脑现在也没人能操作。****

那么遗憾的是：

****目前不能仅仅靠登录同一个 ChatGPT 账号，把旧电脑里的 Codex 本地聊天直接恢复到新电脑。****

因为这些本地 Codex session/history 本身保存在旧电脑上，而不是普通 ChatGPT 那种自动跨设备聊天历史。□cite□turn887245view3□turn887245view0□

这种情况下，你现在可以通过 GitHub 把****代码****恢复到电脑 B：

```
git clone ...
```

但旧电脑上的：

```
Codex conversation
Codex local session state
  worktree state
  push
```

如果没有上传/同步/Remote，就还留在电脑 A。

等你重新拿到电脑 A 后，再设置 Remote/Hand off 是最稳妥的。

不建议直接 OneDrive 同步整个 `~/.codex`

你可能会想到：

```
A ~/.codex
  ↓
OneDrive
  ↓
B ~/.codex
```

****我不建议这么做。****

官方说明 `~/.codex` 不只是聊天历史，它还可能包含：

```
config.toml
auth.json
history.jsonl
logs
cache
  per-user state
```

其中甚至涉及认证信息。□cite□turn887245view3□

所以把：

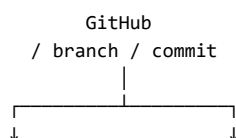
```
~/.codex/*
```

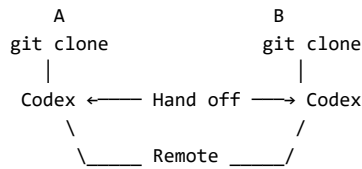
整个扔进 OneDrive / Dropbox /

GitHub，在两台机器同时读写，很容易出现状态冲突，而且还可能涉及凭证安全。更重要的是，****同步 `history.jsonl` 不等于官方支持的 Codex Desktop thread 同步机制。****

对你最适合的长期方案

考虑到你经常用 Codex 修改较大的 Python 项目，我会建议你采用：





其中职责非常清楚：

****GitHub****

→ 同步代码

****Codex Remote****

→ 访问另一台电脑环境

****Codex Hand off****

→ 转移聊天 + Git state

****同一个 ChatGPT account/workspace****

→ 身份和设备授权

这比想办法硬同步 `.codex` 目录可靠得多。官方现在甚至明确把 Hand off 定义为在本地电脑和连接主机之间移动 ****existing chat and its Git state****。[cite]turn887245view1[

所以，****如果你以后经常两台电脑来回切换，我建议不要追求“两份 Codex 记录实时镜像”，而是使用一个 chat，在两台主机之间 Hand off。****这是目前最接近真正“无缝续聊”的官方方式。

你现在自己的电脑不在身边，如果它****之前没有打开 Remote****，那眼下最现实的是先在当前电脑从 GitHub 恢复项目；等你拿回旧电脑后，我建议一次性把 ****GitHub + Codex Remote + Hand off + 两台电脑 repository**** 配置好。以后你在任何一台电脑上做 thesis 的 Stage 1 / Stage 2，都可以把同一个 Codex 线程交接过去继续。[cite]turn887245view1[

用户 2026-08-10 10:57:58

codex app desktop 如何下载聊天记录？

助手 2026-08-10 10:58:13

目前 ****Codex Desktop 没有一个像普通聊天软件那样的“Export this chat / 下载单个聊天”按钮****。OpenAI 官方对 Codex Desktop 的聊天管理目前主要提供 ****Archive / Delete****，没有公开文档说明支持把单个 Codex thread 直接导出成 Markdown、PDF 或 JSON。[cite]turn283844search20[

但你有两种比较实用的办法。

第一种是直接备份 Codex 的****本地会话记录****。OpenAI 官方说明，Codex 默认会把本地 session transcript 保存到 `CODEX\_HOME`，通常就是：

```
~/ .codex/
```

其中历史记录通常包括：

```
~/ .codex/history.jsonl
```

官方明确说明 `history.jsonl` 用于本地历史持久化。[cite]turn283844search1[

如果你是 Windows，可以先在 PowerShell 里看：

```
cd $HOME\ .codex
dir
```

然后备份整个目录，例如：

```
Copy-Item "$HOME\.codex" "D:\Backup\codex-backup" -Recurse
```

如果只是想保存历史文本：

```
Copy-Item "$HOME\.codex\history.jsonl" "D:\Backup\history.jsonl"
```

Mac/Linux 则是：

```
cp -R ~/.codex ~/Desktop/codex-backup
```

这更适合你的目的：****以后换电脑、保存论文项目的 Codex 工作记录，或者做迁移备份。****

不过注意，`~/.codex` 不只是普通聊天文本。Codex 的本地状态、memory 等也会存放在 Codex home 下；例如 Codex memories 默认也在 `~/.codex/memories/`。

所以如果你只是备份，可以复制整个`.codex`；但****不要把整个`.codex`公开上传到 GitHub****，因为其中可能还有账户或本机相关状态。

第二种办法是使用 ****ChatGPT Data Export****。如果你用的是个人 ChatGPT 账号，可以进入：

```
ChatGPT
→ Settings
→ Data Controls
→ Export data
→ Confirm export
```

之后 OpenAI 会发送下载通知，ZIP 中包含 ChatGPT 历史以及其他相关账户数据。

但这里有个重要区别：****Codex history 和普通 ChatGPT history 是分开的****。OpenAI 目前明确说明 Codex 的历史仍与 ChatGPT history 分离。所以如果你的目标是备份 ****Codex Desktop 本地 repository 对应的那些 Codex threads****，我更建议不要只依赖 ChatGPT 的 Data Export，而是直接备份：

```
~/.codex/
```

尤其是你前面问的“两台电脑之间保持 Codex 记录”，最稳妥的是：

```
A
~/.codex/
|
└─┬─
   ↓
External SSD /
```

```
A → GitHub ← B
```

然后让 GitHub 负责代码，`.codex` 负责保存 Codex 本地历史。

如果你愿意，我下一步可以直接告诉你 ****Windows Codex Desktop 的聊天记录具体存在哪些文件夹、哪些文件必须复制，怎么复制到另一台 Windows 电脑后尽量恢复原来的 Codex chats****。

用户 2026-08-10 11:16:43

```
PS C:\Users\XiaoyanXiong.codex> dir
```

```
Directory: C:\Users\XiaoyanXiong\.codex
```

```
Mode LastWriteTime Length Name
```

```
---
```

d----- 8/7/2026 11:21 AM .sandbox
d----- 8/5/2026 1:14 PM .sandbox-bin
d----- 7/7/2026 8:58 AM .sandbox-secrets
d----- 7/7/2026 8:58 AM .tmp
d----- 8/10/2026 9:37 AM archived_sessions
d----- 8/5/2026 11:17 AM attachments
d----- 7/2/2026 11:21 AM cache
d----- 8/7/2026 9:35 AM computer-use
d----- 8/5/2026 10:26 AM dictation-history
d----- 8/7/2026 11:22 AM generated_images
d----- 8/5/2026 10:26 AM pets
d----- 8/10/2026 9:36 AM plugins
d----- 7/2/2026 11:27 AM process_manager
d----- 7/2/2026 11:48 AM rules
d----- 7/2/2026 11:26 AM sessions
d----- 8/10/2026 9:35 AM skills
d----- 8/10/2026 9:37 AM sqlite
d----- 8/10/2026 9:37 AM thread-writer-locks
d----- 3/2/2026 2:09 PM tmp
d----- 7/2/2026 11:19 AM vendor_imports
d----- 7/13/2026 9:30 AM visualizations
-a---- 7/20/2026 7:13 PM 0 ..codex-global-state.json.bak.tmp-1784567583607-79ed9237-b070-4aa4-a2bf-2f32674f6468
-a---- 8/2/2026 3:46 AM 61012
..codex-global-state.json.tmp-1785635174105-816fbda4-6584-4103-a291-39917a20b8fc
-a---- 8/10/2026 11:08 AM 77256 .codex-global-state.json
-a---- 8/10/2026 11:08 AM 77256 .codex-global-state.json.bak
-a---- 7/2/2026 11:19 AM 3 .personality_migration
-a---- 7/29/2026 10:28 AM 3 .sandbox_migration
-a---- 8/10/2026 10:56 AM 0 AGENTS.md
-a---- 8/5/2026 10:25 AM 3976 auth.json
-a---- 8/6/2026 2:46 PM 1364 cap_sid
-a---- 8/10/2026 9:36 AM 3647 config.toml
-a---- 8/10/2026 9:45 AM 32768 goals_1.sqlite
-a---- 7/2/2026 11:19 AM 36 installation_id
-a---- 8/10/2026 11:06 AM 26136576 logs_2.sqlite
-a---- 8/10/2026 11:06 AM 32768 logs_2.sqlite-shm
-a---- 8/10/2026 11:06 AM 4152 logs_2.sqlite-wal
-a---- 8/10/2026 9:55 AM 40960 memories_1.sqlite
-a---- 8/10/2026 11:09 AM 197172 models_cache.json
-a---- 8/6/2026 2:46 PM 1334 session_index.jsonl
-a---- 8/10/2026 10:35 AM 1097728 state_5.sqlite
-a---- 8/10/2026 10:55 AM 32768 state_5.sqlite-shm
-a---- 8/10/2026 10:56 AM 8272 state_5.sqlite-wal
-a---- 8/10/2026 9:43 AM 0 transcription-history.jsonl

那个是聊天记录?

助手 2026-08-10 11:16:58

从你这个新版 Codex Desktop 目录来看，**聊天记录主体就是这两个目录**：

```
C:\Users\XiaoyanXiong\.codex\sessions
C:\Users\XiaoyanXiong\.codex\archived_sessions
```

官方现在明确说明：Codex 的每个持久化 thread/chat transcript 是**保存在磁盘上的 JSONL 文件**；当你 Archive 一个聊天时，Codex 会把对应的 thread log 从 active sessions 目录移动到 archived sessions 目录。 [CiteTurn626843view0](#)

所以你可以这样理解：

```
| 文件/目录 | 作用 |
|---|---|
| `sessions\` | □ 当前未归档的 Codex 聊天记录 |
| `archived_sessions\` | □ 已归档的聊天记录 |
| `session_index.jsonl` | 会话索引/元数据，帮助 Codex 找到 session |
| `.codex-global-state.json` | Desktop App 的全局 UI / 状态信息，不是完整聊天正文 |
| `state_5.sqlite` | App 状态数据库，可能包含 thread/project 元数据，但不是主要 transcript |
| `logs_2.sqlite` | 日志/运行信息，不建议当聊天记录备份 |
| `memories_1.sqlite` | memory 数据，不是聊天 transcript |
| `attachments\` | 聊天使用过的附件 |
| `auth.json` | □□ 登录认证信息，不是聊天记录，绝对不要分享 |
| `config.toml` | Codex 配置 |
```

官方也明确区分了“Archive”和“Delete”：Archive 会保留 transcript；Delete 则会永久删除保存的 session transcript。 [CiteTurn626843view1](#)

你现在可以马上查看 `sessions`

PowerShell 运行：

```
Get-ChildItem "$HOME\.codex\sessions" -Recurse
```

或者只找 JSONL：

```
Get-ChildItem "$HOME\.codex\sessions" -Recurse -Filter *.jsonl
```

你大概率会看到类似这种结构：

```
.codex
├── sessions
│   ├── 2026
│   │   ├── 08
│   │   │   ├── 10
│   │   │   │   ├── rollout-2026-08-10T09-35-....jsonl
│   │   │   │   ├── rollout-2026-08-10T10-02-....jsonl
│   │   │   │   └── ...
│   └── ...
```

这些 `jsonl` 文件才是你最关心的 **Codex conversation transcript**。

可以直接看看其中一个文件：

```
Get-Content " \xxx.jsonl" -First 20
```

或者找最近修改的聊天：

```
Get-ChildItem "$HOME\.codex\sessions" -Recurse -Filter *.jsonl |
    Sort-Object LastWriteTime -Descending |
    Select-Object -First 20 FullName, LastWriteTime, Length
```

这样非常适合你定位：

> “今天上午那个 Codex 聊天到底是哪一个文件？”

`session\_index.jsonl` 也值得保留

你的：

```
C:\Users\XiaoyanXiong\.codex\session_index.jsonl
```

只有 1334 bytes，所以它显然**不可能保存所有聊天正文**。

它更像：

```
    ID
  ↓
  ↓
session
```

真正大量的 conversation content 在 `sessions` 下面的 JSONL transcript。

你可以直接看看：

```
Get-Content "$HOME\.codex\session_index.jsonl"
```

可能会看到 session UUID、name/title 等信息。

`archived\_sessions` 是什么？

你这个目录：

```
8/10/2026 9:37 AM archived_sessions
```

正好非常符合 Codex 当前 Archive 机制。

官方 App Server 文档明确写的是：

> `thread/archive` 会把持久化的 JSONL thread log 移动到 **archived sessions directory**。[CiteTurn626843view0](#)

所以：

```
sessions
  ↓ Archive
archived_sessions
```

反过来 Unarchive：

```
archived_sessions
  ↓ Unarchive
sessions
```

官方现在也支持：

```
codex archive <SESSION>
codex unarchive <SESSION>
codex resume <SESSION>
```

其中 `codex resume` 可以继续之前保存的 session。[CiteTurn626843view1](#)

如果你真正目的是 “备份 Codex 聊天”

我建议至少备份这三个：

把输出贴给我，我可以进一步帮你判断**哪个 JSONL 对应哪个 Codex Desktop 聊天，并告诉你怎么把所有这些聊天迁移到另一台电脑，让 Codex Desktop 尽可能原样显示出来**。